Problem 1: Retail Sales Order and Customer Loyalty System

Overview of the System :-

The Retail Sales Order and Customer Loyalty System is a Python-based application developed using Object-Oriented Programming (OOP) concepts. The system is designed to manage online product sales, customer orders, loyalty benefits, payment processing, stock management, and order record storage in an organized and secure manner.

This system supports different customer categories such as Regular Customers, Loyalty Customers, and Business Customers. Each customer type receives different discounts and benefits based on predefined business rules. Customers can place orders only when sufficient product stock is available and payment is successfully completed.

The system ensures proper payment and stock handling. Product stock is reduced only after successful payment confirmation. If payment fails due to insufficient wallet balance or any other issue, the stock remains unchanged. Every order, including successful and failed orders, is stored in a file for future business analysis and reporting purposes.

The project also generates useful reports such as:

- Total successful orders
- Total failed orders
- Total revenue collected
- Most ordered product
- Failure reason summary

This project demonstrates practical implementation of important Python concepts such as Classes and Objects, Inheritance, Polymorphism, Encapsulation, Abstraction, Exception Handling, and File Handling. The overall system is designed in a realistic way similar to backend logic used in actual e-commerce platforms.

Explanation of the Assignment :-

This project simulates a simple online retail shopping system where customers can purchase products using their wallet balance. Different customer types receive different discounts and benefits according to their category. The system checks product availability, validates order quantity, processes payment, updates stock, and records order details.

Inheritance and method overriding are used to implement different discount rules for customer types. Encapsulation is used to protect sensitive information such as wallet balance, product price, and stock quantity. Abstraction is used to simplify the order placement process by hiding internal operations from the user.

The system also handles invalid situations such as:

- Insufficient stock
- Invalid quantity
- Invalid product price
- Insufficient wallet balance
- Incorrect customer input

All order details are saved into a file so that sales data can later be used for reporting and business analysis. Overall, the project provides a practical and organized implementation of a Retail Sales Order and Customer Loyalty System using Python and OOP principles.

A6 -Practice Problems 1

May 26, 2026

1 Problem 1 : Retail Sales Order and Customer Loyalty System

[ ]:

2 CREATING CUSTOMER CLASS

[1]:
```
class Customer:
    def __init__(self, name, city, wallet_balance):
        self.name=name
        self.city=city
        self.__wallet_balance = wallet_balance # its a private variable

    def get_wallet_balance(self): # to get the wallet balance
        return self.__wallet_balance

    def pay_from_wallet(self, amount):  # Wallet payment
        if amount <= self.__wallet_balance:
            self.__wallet_balance -= amount
            return True
        else:
            return False

    def get_discount_percentage(self): # this is for regular customer discount
        return 0
```

[ ]:

3 creating loyalty customer class with 10%

[2]:
```
class LoyaltyCustomer(Customer):
    def get_discount_percentage(self):  # Loyalty customer gets 10% discount
        return 10
```

[ ]:

4 creating business customer class with 20%

[3]:
```
class BusinessCustomer(Customer):
    def get_discount_percentage(self):
        return 20
```

[ ]:
```

```

5 creaing the product class

[4]:
```
class Product:

    def __init__(self, product_name, price, stock):
        self.product_name = product_name
        self.__price = price
        self.__stock = stock

    def get_price(self):
        return self.__price

    def get_stock(self):
        return self.__stock

    def reduce_stock(self, quantity):
        if quantity <= self.__stock:
            self.__stock -= quantity
            return True
        else:
            return False

    def display_product_details(self):
        print("\nProduct Details")
        print("Product Name:", self.product_name)
        print("Price:", self.__price)
        print("Stock:", self.__stock)
```

[ ]:
```

```

6 creating the order class

[5]:
```
class Order:

    def __init__(self, customer, product, quantity):
        self.customer = customer
        self.product = product
        self.quantity = quantity
```

```
        self.total_amount = 0
        self.discount_amount = 0
        self.final_amount = 0
        self.order_status = "Pending"
        self.failure_reason = "None"

    def place_order(self):
        try:
            quantity = int(self.quantity)
            if quantity <= 0:
                self.order_status = "Failed"
                self.failure_reason = ("Invalid Quantity")
                return
            if quantity > self.product.get_stock():
                self.order_status = "Failed"
                self.failure_reason = ("Stock Not Available")
                return
            self.total_amount = (self.product.get_price() * quantity)
            discount_percentage = (self.customer.get_discount_percentage())
            self.discount_amount = (self.total_amount * discount_percentage /␣
↪100)
            self.final_amount = (self.total_amount - self.discount_amount)
            payment_successful = (self.customer.pay_from_wallet(self.
↪final_amount))

            if not payment_successful:
                self.order_status = "Failed"
                self.failure_reason = ("Low Wallet Balance")
                return
            self.product.reduce_stock(quantity)
            self.order_status = "Order Placed"

        except ValueError:
            self.order_status = "Failed"
            self.failure_reason = ("Quantity Is Not A Number")

    def display_order_details(self):
        print("\nOrder Details")
        print("Customer Name:", self.customer.name)
        print("Product Name:", self.product.product_name)
        print("Quantity:", self.quantity)
        print("Total Amount:", self.total_amount)
        print("Discount Amount:", self.discount_amount)
        print("Final Amount:", self.final_amount)
        print("Order Status:", self.order_status)
        print("Failure Reason:", self.failure_reason)
```

[ ]:

7 Order File Manager Class

[6]:
```python
class OrderFileManager:

    def __init__(self, file_name):
        self.file_name = file_name

    def save_order(self, order):
        with open(self.file_name, "a") as file:
            file.write("Customer Name: "+ order.customer.name + "\n")
            file.write("Product Name: "+ order.product.product_name + "\n")
            file.write("Quantity: "+ str(order.quantity) + "\n")
            file.write("Final Amount: "+ str(order.final_amount) + "\n")
            file.write("Order Status: "+ order.order_status + "\n")
            file.write("Failure Reason: "+ order.failure_reason + "\n")
            file.write("----------------------\n")
        print("Order saved successfully.")
```

[ ]:

8 Report Manager Class

[7]:
```python
class ReportManager:

    def generate_report(self, orders):
        successful_orders = 0
        failed_orders = 0
        total_revenue = 0
        failure_summary = {}
        product_count = {}

        for order in orders:
            if order.order_status == "Order Placed":
                successful_orders += 1
                total_revenue += (order.final_amount)
                product_name = (order.product.product_name)

                if product_name in product_count:
                    product_count[product_name] += (order.quantity)
                else:
                    product_count[product_name] = (order.quantity)

            else:
                failed_orders += 1
```

```
                reason = order.failure_reason
                if reason in failure_summary:
                    failure_summary[reason] += 1
                else:
                    failure_summary[reason] = 1

        most_ordered_product = "None"

        if len(product_count) > 0:
            most_ordered_product = max(product_count,key=product_count.get)

        # for display report
        print("FINAL SALES REPORT------")
        print("Total Successful Orders:",successful_orders)
        print("Total Failed Orders:",failed_orders)
        print("Total Revenue:",total_revenue)
        print("Most Ordered Product:",most_ordered_product)
        print("Failure Reason Summary:",failure_summary)
```

[ ]:

9 Creating Customer Objects

[8]:
```
customer1 = Customer("Deepak","Lucknow",1000)
customer2 = LoyaltyCustomer("Daya","Gorakhpur",1000)
customer3 = BusinessCustomer("Hemant","Delhi",500)
```

[ ]:

10 Creating Product Objects

[9]:
```
product1 = Product("Keyborad",500,10)
product2 = Product("Mouse",200,2)

product1.display_product_details()
product2.display_product_details()
```

```
Product Details
Product Name: Keyborad
Price: 500
Stock: 10

Product Details
Product Name: Mouse
Price: 200
```

```
Stock: 2
```

[ ]:

11 1: making successful order

[10]:
```
order1 = Order(customer1,product1,1)
order1.place_order()
order1.display_order_details()
```

```
Order Details
Customer Name: Deepak
Product Name: Keyborad
Quantity: 1
Total Amount: 500
Discount Amount: 0.0
Final Amount: 500.0
Order Status: Order Placed
Failure Reason: None
```

[ ]:

12 2: For Loyalty Customer Order

[11]:
```
order2 = Order(customer2,product1,1)
order2.place_order()
order2.display_order_details()
```

```
Order Details
Customer Name: Daya
Product Name: Keyborad
Quantity: 1
Total Amount: 500
Discount Amount: 50.0
Final Amount: 450.0
Order Status: Order Placed
Failure Reason: None
```

[ ]:

13 3: Failed order due to low wallet balance

[12]:
```
order3 = Order(customer3,product1,2)
order3.place_order()
order3.display_order_details()
```

```
Order Details
Customer Name: Hemant
Product Name: Keyborad
Quantity: 2
Total Amount: 1000
Discount Amount: 200.0
Final Amount: 800.0
Order Status: Failed
Failure Reason: Low Wallet Balance
```

[ ]:

14 4: failed order due to stock not available

[13]:
```
order4 = Order(customer1,product2,10)
order4.place_order()
order4.display_order_details()
```

```
Order Details
Customer Name: Deepak
Product Name: Mouse
Quantity: 10
Total Amount: 0
Discount Amount: 0
Final Amount: 0
Order Status: Failed
Failure Reason: Stock Not Available
```

[ ]:

15 5: Failed Order Due To Invalid Text Quantity

[14]:
```
order5 = Order(customer1,product1,"one")
order5.place_order()
order5.display_order_details()
```

```
Order Details
Customer Name: Deepak
Product Name: Keyborad
```

```
Quantity: one
Total Amount: 0
Discount Amount: 0
Final Amount: 0
Order Status: Failed
Failure Reason: Quantity Is Not A Number
```

[ ]:

16 6: Failed Order Due To Zero Quantity

[15]:
```
order6 = Order(customer1,product1,0)
order6.place_order()
order6.display_order_details()
```

```
Order Details
Customer Name: Deepak
Product Name: Keyborad
Quantity: 0
Total Amount: 0
Discount Amount: 0
Final Amount: 0
Order Status: Failed
Failure Reason: Invalid Quantity
```

[ ]:

17 7: Saving orders into orders.txt

[16]:
```
file_manager = OrderFileManager("orders.txt")
orders = [order1,order2,order3, order4,order5,order6]

for order in orders:
    file_manager.save_order(order)
```

```
Order saved successfully.
Order saved successfully.
Order saved successfully.
Order saved successfully.
Order saved successfully.
Order saved successfully.
```

[ ]:

18 8: Generating total sales report

[17]:
```
report = ReportManager()
report.generate_report(orders)
```

```
FINAL SALES REPORT------
Total Successful Orders: 2
Total Failed Orders: 4
Total Revenue: 950.0
Most Ordered Product: Keyborad
Failure Reason Summary: {'Low Wallet Balance': 1, 'Stock Not Available': 1,
'Quantity Is Not A Number': 1, 'Invalid Quantity': 1}
```

[ ]:

[ ]:

[ ]: